

無線遙控車系統開發專案

Wireless RC Car System Development

技術專案報告

2026 年 3 月

專案概述

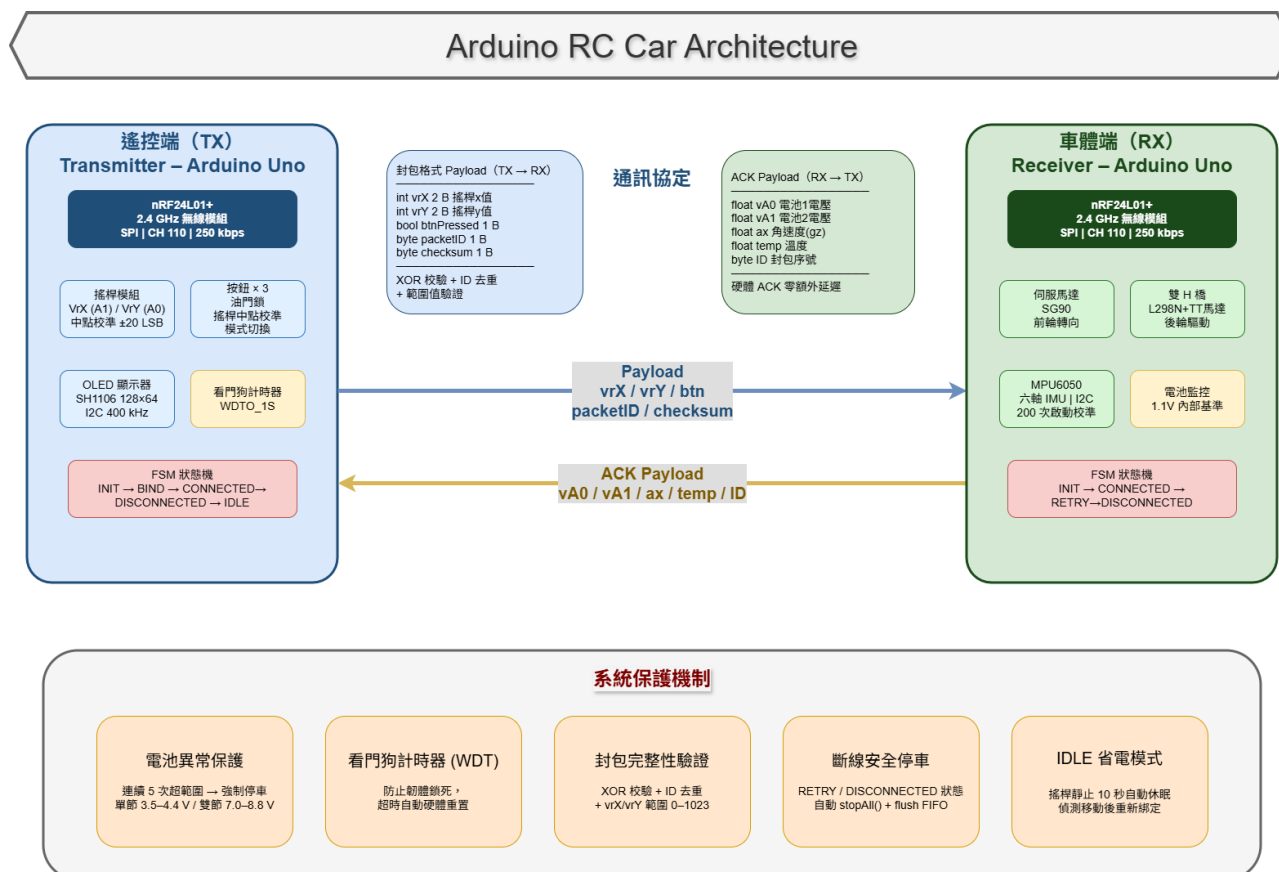
本專案開發一套完整的雙 Arduino 無線遙控車系統，涵蓋遙控端（發射器）與車體端（接收器）兩套韌體。系統採用 nRF24L01+ 2.4GHz 無線模組進行雙向通訊，整合 MPU6050 六軸慣性測量單元、OLED 顯示器、電池電壓監控及伺服馬達轉向控制，具備完善的通訊品質管理與故障保護機制。

項目	規格 / 數值
主控平台	Arduino Uno / Nano (ATmega328P)
無線模組	nRF24L01+ · 頻道 110 · 速率 250 kbps · PA_LOW
通訊延遲目標	< 40 ms (TX 週期 40 ms)
轉向機構	伺服馬達，最大範圍 $\pm 35^{\circ}$
驅動馬達	雙 H 橋 (L298N) · PWM 速度控制
姿態感測	MPU6050 · 200 次校準取平均偏移量
顯示介面	SH1106 128×64 OLED (U8g2 Library)
電池監控	雙節 18650 · 內部 1.1V 參考 · 11:1 分壓
看門狗計時器	WDTO_1S (1 秒硬體重置保護)

系統架構

2.1 整體架構

系統由遙控端與車體端組成，兩端透過 nRF24L01+ 進行雙向通訊。遙控端為 TX 模式，負責讀取搖桿輸入、打包資料並發送；車體端為 RX 模式，接收指令後驅動馬達與轉向，同時利用 nRF24L01 的 ACK Payload 機制回傳感測器狀態。



2.2 通訊協定設計

通訊採用自定義封包格式，加入封包 ID 與 XOR 校驗，確保資料完整性並過濾重複封包：

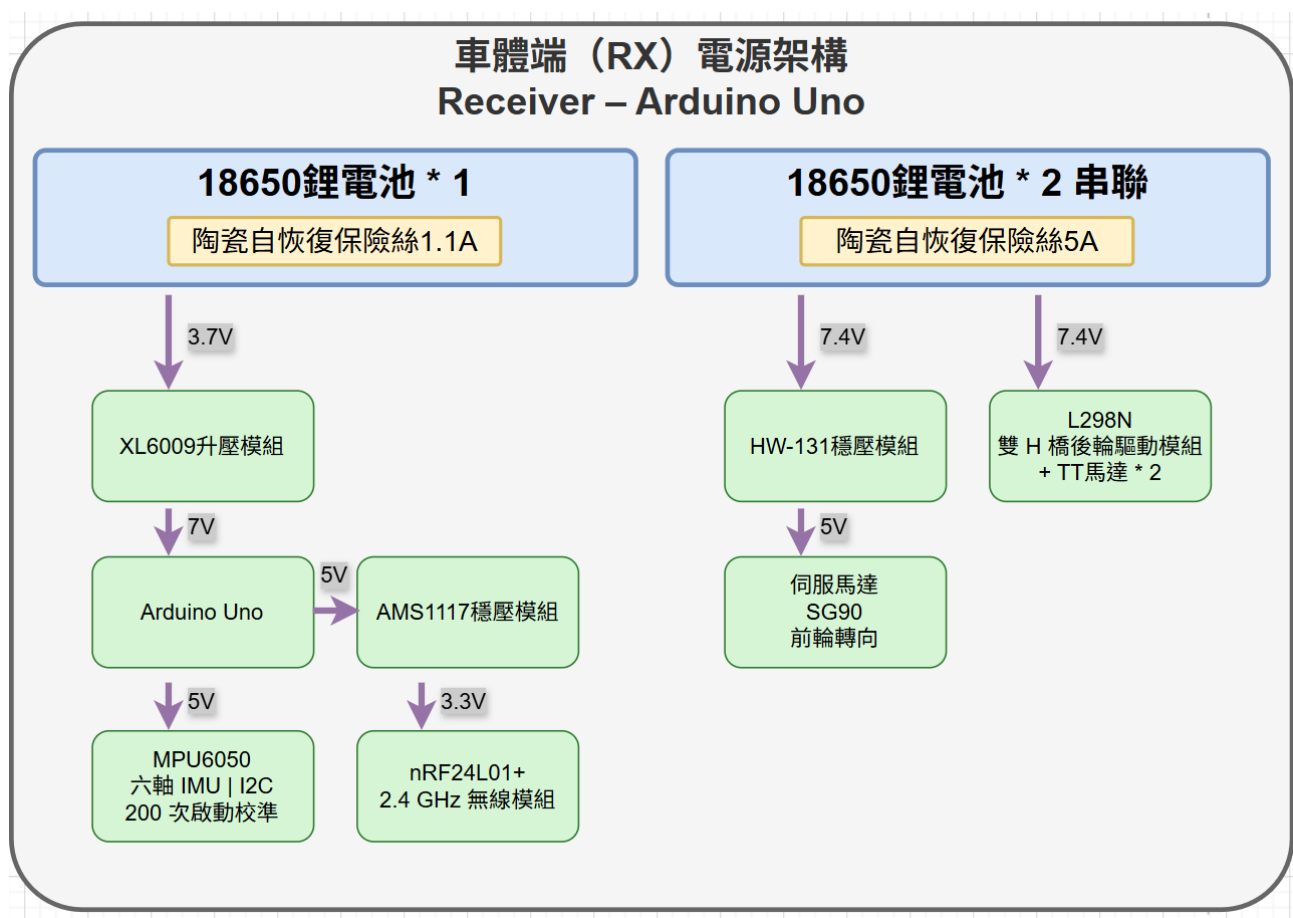
欄位	型別	大小	說明
vrX	int	2 Bytes	搖桿 X 軸 (0~1023)
vrY	int	2 Bytes	搖桿 Y 軸 (0~1023)
btnPressed	bool	1 Byte	按鈕狀態
packetID	byte	1 Byte	遞增封包序號 (防重複)
checksum	byte	1 Byte	XOR 校驗 : $vrX \wedge vrY \wedge ID \wedge btn$

2.3 ACK 回傳資料

車體端利用 nRF24L01 的 ACK Payload 機制，在硬體確認封包中同步回傳車體狀態，無需額外發送週期：

欄位	型別	說明
vA0 / vA1	float	電池 1 / 電池 2 電壓 (V)
ax	float	MPU6050 角速度 (gz 軸 · rad/s)
temp	float	MPU6050 板載溫度 (°C)
ID	byte	對應封包序號 (用於去重)

2.4 電源架構



核心功能與技術亮點

3.1 有限狀態機通訊管理

兩端韌體均實作有限狀態機 (FSM) 管理無線連線品質，確保斷線時車體自動停止、重連後平滑恢復：

狀態	車體端觸發條件	動作
INIT	長時間 DISCONNECTED 後	重新初始化 Radio
CONNECTED	收到有效封包	正常驅動，執行 driveByJoystick()
RETRY	250 ms 未收到封包	停車，清除 RX/TX FIFO
DISCONNECTED	RETRY 持續 500 ms	停車，關閉並重開 Pipe

遙控端另增 IDLE 狀態：搖桿 10 秒未移動自動進入休眠，偵測到移動後重新綁定，有效降低不必要的射頻發射。

3.2 資料完整性驗證

車體端對每個接收封包執行三重驗證，防止雜訊或中途干擾導致錯誤動作：

- XOR 校驗和比對，確保位元層完整性
- 封包 ID 去重，過濾重複或亂序封包
- 數值範圍檢查 (vrX / vrY 需在 0–1023 之間)

3.3 MPU6050 慣性感測與校準

系統啟動時對 MPU6050 執行 200 次採樣校準，計算各軸靜態偏移量，後續量測均自動補償。感測資料 (角速度、溫度) 每 300 ms 更新一次，透過 ACK Payload 回傳至遙控端顯示，為未來實作陀螺儀輔助穩定控制預留介面。

3.4 電池健康監控

採用 ATmega328P 內部 1.1V 基準電壓搭配 11:1 分壓電路，量測 18650 鋰電池電壓，並以軟體監控：

- 連續 5 次量測超出正常範圍 (單節 3.5~4.4 V , 雙節 7.0~8.8 V) , 觸發強制停車
- 量測結果透過 ACK 回傳至 OLED 即時顯示於遙控器端 , 方便監控
- 依據電表量測比對 , 加入電壓校正偏移 ($\pm 0.2\text{ V}$) 補償分壓電阻誤差

3.5 遙控器搖桿中點校準

遙控端提供搖桿中點校正功能：按下 BTN_pin3 即讀取當前搖桿原始值計算偏移，校正值限制在 ± 20 LSB 範圍內，防止異常數值影響控制，校正間隔至少 1 秒以防誤觸。

3.6 驅動模式切換

遙控端支援三種驅動模式 (A / B / C) 循環切換，模式即時顯示於 OLED。模式 A 為全功能搖桿控制，模式 B 為低速模式，C 則傳送中立值，可用於未來擴展差速或輔助功能。

實作技術細節

4.1 韌體架構設計

兩端韌體均採用結構體封裝 (struct-based modular design) 取代全域函式，提升可讀性與可維護性：

模組 (Struct)	所在端	主要職責
DriveManager	車體端	伺服馬達轉向、H 橋 PWM 驅動、死區過濾
MPU6050Manager	車體端	I2C 初始化、200 次校準、週期性資料更新
BatteryManager	車體端	ADC 多次取樣、電壓計算、異常計數保護
StateMetrics	兩端	FSM 狀態管理、時間戳記、超時判斷
DisplayManager	遙控端	OLED 靜態 / 動態分離更新 (每 250 ms)
JoyStickManager	遙控端	搖桿讀值、中點校準、封包打包與校驗

4.2 即時性保障

- TX 端主迴圈以 40 ms 固定週期發送，while(millis() - loopStart < TX_PERIOD) 精確等待

- RX 端以 while(`radio.available()`) 清空 FIFO，確保不遺漏封包
- 各感測器與顯示器採獨立更新週期（300 / 500 / 250 ms），避免阻塞主迴圈
- 看門狗計時器 `WDTO_1S` 防止韌體死鎖，確保系統可靠運行

4.3 OLED 顯示優化

OLED 採靜態 / 動態分離策略：標題與標籤僅在 `setup()` 時寫入一次，主迴圈僅更新數值欄位，大幅降低 I2C 傳輸量，減少畫面閃爍並節省 CPU 時間。

技術能力展示

技術類別	具體應用
嵌入式 C++	結構體模組化設計、位元操作、packed struct 封包序列化
無線通訊	nRF24L01+ SPI 驅動、雙向 ACK Payload、動態 Payload
感測器整合	MPU6050 I2C 驅動、六軸資料讀取與偏移校準
馬達控制	H 橋 PWM 驅動、伺服馬達 PWM、死區過濾、速度映射
系統設計	有限狀態機 (FSM)、看門狗計時器 (WDT)、模組化韌體架構
電源管理	ADC 內部基準、分壓量測、電池異常保護邏輯
人機介面	OLED SH1106 驅動、U8g2 Library、靜態 / 動態分區更新

總結

本專案完整實現從底層硬體驅動到應用層通訊協定的嵌入式開發，展示了在資源受限的 MCU 環境下設計穩健、可維護系統的能力。專案涵蓋無線通訊、感測器融合、即時控制與故障保護等核心嵌入式技術，適合作為嵌入式軟體工程師、韌體工程師及物聯網應用開發職位的技術佐證。